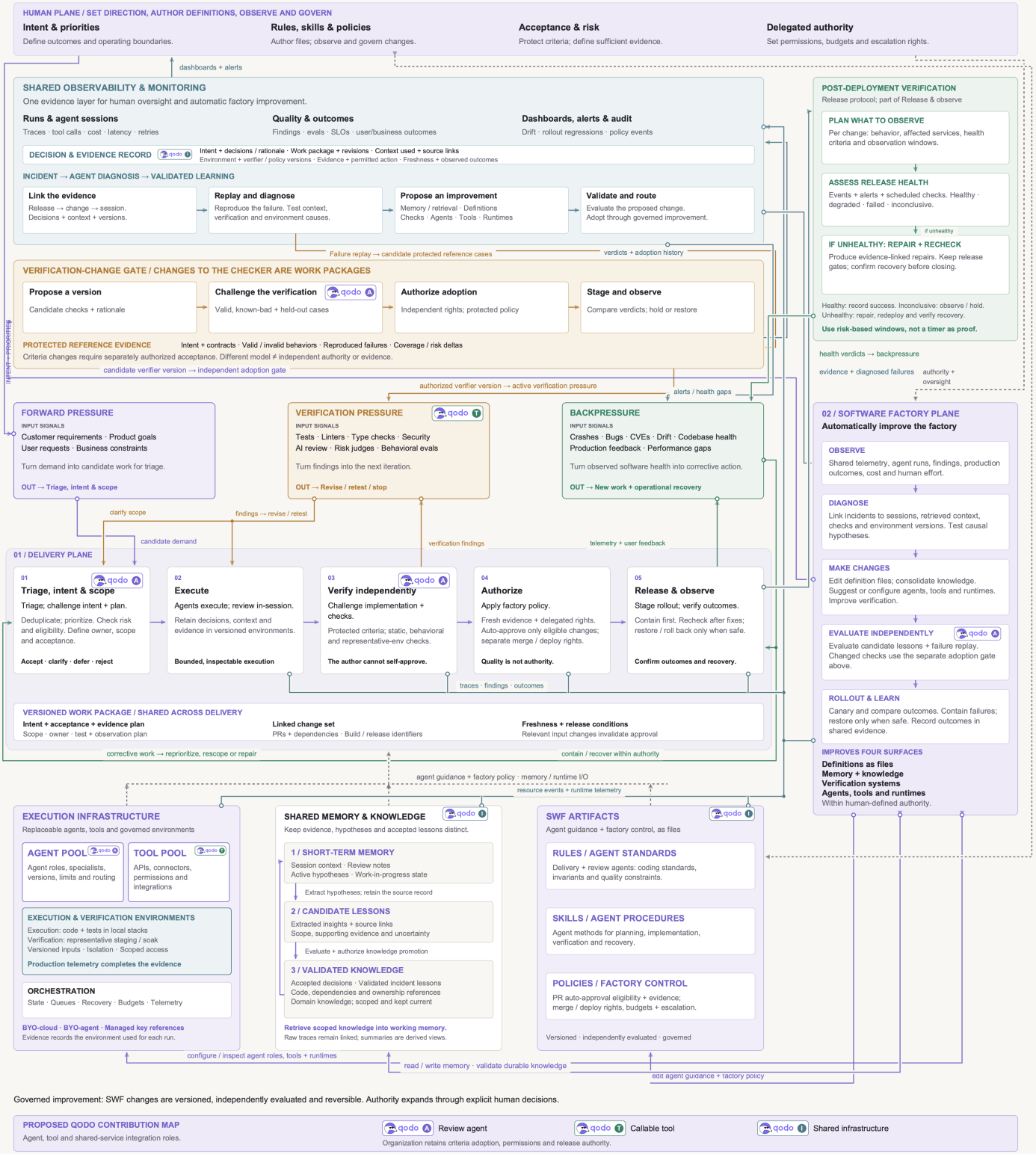
Software factory architecture | v14

A4 print edition. Page 1 is the overview; pages 2-6 enlarge the content for reading. Digital map: 3420 x 3940.

Software factory architecture

Qodo overlay: proposed agent, tool and shared-service contributions. Organizational authority stays explicit.



Print at actual size. Badges identify proposed agent, tool and shared-infrastructure roles. Organizations retain authority.

Human authority, evidence and learning

The organization defines the boundaries; delivery and factory improvement share the same evidence layer.

Human plane

Intent & priorities
Define outcomes and operating boundaries.

Rules, skills & policies
Author files; observe and govern changes.

Acceptance & risk
Protect criteria; define sufficient evidence.

Delegated authority
Set permissions, budgets and escalation rights.

Shared observability and monitoring

One evidence layer for human oversight and automatic factory improvement.

Runs & agent sessions
Traces · tool calls · cost · latency · retries

Quality & outcomes
Findings · evals · SLOs · user/business outcomes

Dashboards, alerts & audit
Drift · rollout regressions · policy events

Decision and evidence record [I]
Intent + decisions / rationale · Work package + revisions · Context used + source links
Environment + verifier / policy versions · Evidence + permitted action · Freshness + observed outcomes

Incident -> agent diagnosis -> validated learning

Link the evidence
Release → change → session.
Decisions + context + versions.

Replay and diagnose
Reproduce the failure. Test context, verification and environment causes.

Propose an improvement
Memory / retrieval · Definitions
Checks · Agents · Tools · Runtimes

Validate and route
Evaluate the proposed change.
Adopt through governed improvement.

Failure replay supplies candidate protected reference cases. Diagnosed failures and evidence feed the SWF improvement plane.

Verification-change gate

Propose a version
Candidate checks + rationale

Challenge the verification [A]
Valid, known-bad + held-out cases

Authorize adoption
Independent rights; protected policy

Stage and observe
Compare verdicts; hold or restore

Protected reference evidence: Intent + contracts · Valid / invalid behaviors · Reproduced failures · Coverage / risk deltas

Criteria changes require separately authorized acceptance. Different model ≠ independent authority or evidence.

Only the authorized verifier version becomes active verification pressure. Verdicts and adoption history return to shared observability.

Pressures and the delivery plane

Triage is part of the first delivery step, before execution. Authorization applies factory policy to fresh evidence.

Three sources of pressure

FORWARD PRESSURE

INPUT SIGNALS

Customer requirements · Product goals
User requests · Business constraints

Turn demand into candidate work for triage.

OUT → Triage, intent & scope

VERIFICATION PRESSURE [T]

INPUT SIGNALS

Tests · Linters · Type checks · Security
AI review · Risk judges · Behavioral evals

Turn findings into the next iteration.

OUT → Revise / retest / stop

BACKPRESSURE

INPUT SIGNALS

Crashes · Bugs · CVEs · Drift · Codebase health
Production feedback · Performance gaps

Turn observed software health into corrective action.

OUT → New work + operational recovery

Delivery flow: 01 -> 02 -> 03 -> 04 -> 05

Triage, intent & scope [A]

Triage; challenge intent + plan.

Deduplicate; prioritize. Check risk and eligibility. Define owner, scope and acceptance.

Accept · clarify · defer · reject

Execute

Agents execute; review in-session.

Retain decisions, context and evidence in versioned environments.

Bounded, inspectable execution

Verify independently [A]

Challenge implementation + checks.

Protected criteria; static, behavioral and representative-env checks.

The author cannot self-approve.

Authorize

Apply factory policy.

Fresh evidence + delegated rights. Auto-approve only eligible changes; separate merge / deploy rights.

Quality is not authority.

Release & observe

Stage rollout; verify outcomes.

Contain first. Recheck after fixes; restore / roll back only when safe.

Confirm outcomes and recovery.

Versioned work package, shared across delivery

Intent + acceptance + evidence plan

Scope · owner · test + observation plan

Linked change set

PRs + dependencies · Build / release identifiers

Freshness + release conditions

Relevant input changes invalidate approval

Forward pressure supplies candidate demand. Verification findings return work to execution or scope clarification. Production feedback creates corrective work and can trigger containment or recovery within delegated authority.

Relevant input changes invalidate approval. Quality evidence does not itself grant authority to merge or deploy.

The shared resources

Execution infrastructure is on the left, memory in the middle, and SWF artifacts beside the factory-improvement plane.

Execution infrastructure

AGENT POOL [A]

Agent roles, specialists,
versions, limits and routing

TOOL POOL [T]

APIs, connectors,
permissions and
integrations

EXECUTION & VERIFICATION ENVIRONMENTS

Execution: code + tests in local stacks
Verification: representative staging / soak
Versioned inputs · Isolation · Scoped access

Production telemetry completes the evidence

ORCHESTRATION

State · Queues · Recovery · Budgets · Telemetry

BYO-cloud · BYO-agent · Managed key references

Evidence records the environment used for each run.

Local execution tests the code in a development stack. Representative verification exercises the relevant dependencies, configuration, data conditions and cross-service behavior. Production observations supply further evidence. Delivery, reviewers and factory evaluations use the appropriate environment.

Shared memory and knowledge [I]

1 / SHORT-TERM MEMORY

Session context · Review notes
Active hypotheses · Work-in-progress state

2 / CANDIDATE LESSONS

Extracted insights + source links
Scope, supporting evidence and uncertainty

3 / VALIDATED KNOWLEDGE

Accepted decisions · Validated incident lessons
Code, dependencies and ownership references
Domain knowledge; scoped and kept current

Short-term memory -> extract hypotheses with source links -> candidate lessons -> evaluate and authorize promotion -> validated knowledge. Retrieve scoped knowledge into working memory.

Raw traces remain linked; summaries are derived views.

SWF artifacts [I]

RULES / AGENT STANDARDS

Delivery + review agents: coding standards,
invariants and quality constraints.

SKILLS / AGENT PROCEDURES

Agent methods for planning, implementation,
verification and recovery.

POLICIES / FACTORY CONTROL

PR auto-approval eligibility + evidence;
merge / deploy rights, budgets + escalation.

Versioned · independently evaluated · governed

Rules and skills guide delivery and review agents. Factory policies govern the workflow, including PR auto-approval eligibility, required evidence, merge/deploy rights, budgets and escalation. Humans author and govern these files; the SWF plane improves them within delegated authority.

The software factory improvement plane

The second job of the factory is to improve how future work gets done.

Observe -> Diagnose -> Make changes -> Evaluate -> Rollout and learn

OBSERVE

Shared telemetry, agent runs, findings, production outcomes, cost and human effort.

DIAGNOSE

Link incidents to sessions, retrieved context, checks and environment versions. Test causal hypotheses.

MAKE CHANGES

Edit definition files; consolidate knowledge. Suggest or configure agents, tools and runtimes. Improve verification.

EVALUATE INDEPENDENTLY [A]

Evaluate candidate lessons + failure replay. Changed checks use the separate adoption gate above.

ROLLOUT & LEARN

Canary and compare outcomes. Contain failures; restore only when safe. Record outcomes in shared evidence.

Four improvement surfaces

Definitions as files; memory and knowledge; verification systems; agents, tools and runtimes. Changes remain within human-defined authority.

Connections and decision rights

Human authority

Humans define intent, acceptance, risk and delegated authority. They author, observe and govern the artifact files.

Shared evidence

Delivery runs, quality outcomes and resource/runtime events feed observability. Humans and the SWF plane consume the same evidence.

Resource interfaces

Memory and execution infrastructure have bidirectional interfaces: context retrieval and updates, calls and results. Delivery consumes agent guidance and factory policy.

Governed changes

The SWF plane edits guidance and policy, validates durable knowledge, and configures or inspects agent roles, tools and runtimes. Changed verification uses the separate adoption gate.

SWF changes are versioned, independently evaluated and reversible. Authority expands through explicit human decisions.

Proposed Qodo contribution map

[A] Review agent [T] Callable tool [I] Shared infrastructure

Agent, tool and shared-service integration roles. The organization retains criteria adoption, permissions and release authority.

Post-deployment verification

Observe every release; verify recovery when corrective action is needed.

Portable evidence plans belong to the work package

Verification plan

State the expected behavior, failure cases, affected services and required checks. Bind results to the tested revisions, environment and acceptance criteria.

Observation plan

Specify what to watch after release, the relevant baseline and health thresholds, the observation window, and which findings require containment or human judgment.

Keep these plans as versioned, text-readable artifacts usable by different agents and tools. The plan describes required evidence; passing a plan review does not itself authorize deployment.

The release-verification protocol

PLAN WHAT TO OBSERVE

Per change: behavior, affected services, health
criteria and observation windows.

ASSESS RELEASE HEALTH

Events + alerts + scheduled checks. Healthy ·
degraded · failed · inconclusive.

IF UNHEALTHY: REPAIR + RECHECK

Produce evidence-linked repairs. Keep release
gates; confirm recovery before closing.

A release and its evidence enter the protocol. Health verdicts feed backpressure; observations and decisions enter the shared evidence record. Monitoring continues after corrective deployments.

Health verdicts remain distinct

Healthy

The expected outcomes meet the declared criteria, supported by sufficient fresh observations for the relevant window.

Degraded

Outcomes have worsened. Limit exposure and create corrective work according to the applicable risk policy.

Failed

A required health or acceptance criterion is breached. Hold, halt or isolate within delegated authority; restore or roll back only when safe.

Inconclusive

Evidence is missing, stale or conflicting. Continue observation, hold the next action, or escalate. Uncertainty does not count as success.

Durable monitoring and repair closure

Deployment events, alerts and scheduled checks should refer to the release and environment being assessed. Persist monitoring state, handle duplicate events and retries, and retain the observations behind each verdict. Observation windows depend on the change, its risk and the behavior being verified; elapsed time alone proves nothing.

A repair creates linked corrective work. After the authorized fix is deployed, repeat the relevant checks and observe the outcome. Close the incident only when the evidence confirms recovery. Record the decisions, rationale, context and unresolved uncertainty along the way.

Authority remains explicit

Detection, assessment and repair generation do not automatically grant merge or deployment rights. Apply the declared policy at every consequential step. Contain failures before relying on timely human intervention.